

```
// app/src/main/java/com/example/appname/App.java
import android.app.Application;
public class App extends Application {

    @Override
    public void onCreate() {
        super.onCreate();
        AppController.attach(this);
    }
}
```

If you need flavour-specific class logic, create a folder named with that flavour name. You can also change the default path to flavour-specific code using the `sourceSets` method, but it is better to use the default option. You can use this flavour path to put specific configuration files like JSON files that you get for Google services.

You can conditionally include your dependencies also. In the example, *Stetho* dependency is only needed for your *debug* builds, for which you can use your build variant name with the *compile* method.

```
compile 'com.android.support.customtabs:25.2.0'
// only used on debug builds
debugCompile 'com.facebook.stetho:stetho:1.4.2'
```

Creating flavour-specific resources or configuration values

In your app's Gradle *config* file, you can also specify a flavour- or variant-specific resource value or *config* value. Check the code below:

```
android{

    productFlavors {
        mock {
            resValue "string", "app_name", "Appname-Mock"
        }

        production {
            resValue "string", "app_name", "Appname"
        }
    }

    buildTypes {

        debug{
            buildConfigField "String", "RAMEN_EFFECT", "0"
        }
        release{
            buildConfigField "String", "RAMEN_EFFECT", "1"
        }
    }
}
```

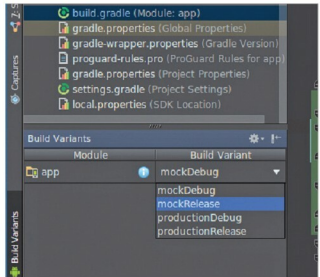


Figure 1: Selecting build types or flavour

In the example, the *resValue* method creates a string resource with the key *app_name* and value based on the type of flavour. *buildConfigField* creates a field in the *BuildConfig* class, and you can access the generated values in your code using *BuildConfig.FIELD_NAME*.

Using the *properties* files for secret data

When you push your code to remote repositories, you may not want to share your API server details or API keys. For such cases, you can make use of the *properties* file and ignore this file in your version control (*.gitignore* file in Git).

```
// app/build.gradle
// tries to read from environment, useful for CI systems
def api_url = System.getenv("API_URL")
```

```
Properties properties = new Properties()
// lookups config.properties file in project root
if (project.rootProject.file('config.properties').exists()) {
    properties.load(project.rootProject.file('config.
properties').newDataInputStream())
    api_url = properties.getProperty("apiUrl1")
}
```

In the above example, the Gradle *config* will read the secrets from a *properties* file (*config.properties*) and assign it to a variable. This variable can be used with the *resValue* or the *buildConfigField* methods in the application code. 

By: Renjith Thankachan

The author works in a startup called Pytenlabs as a product engineer. His areas of expertise are Django/Python, Android and Linux server administration. He can be reached at renjith@pytenlabs.com.